ECE0202
Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute

Overview: Arithmetic and Logic Instructions

- Syntax

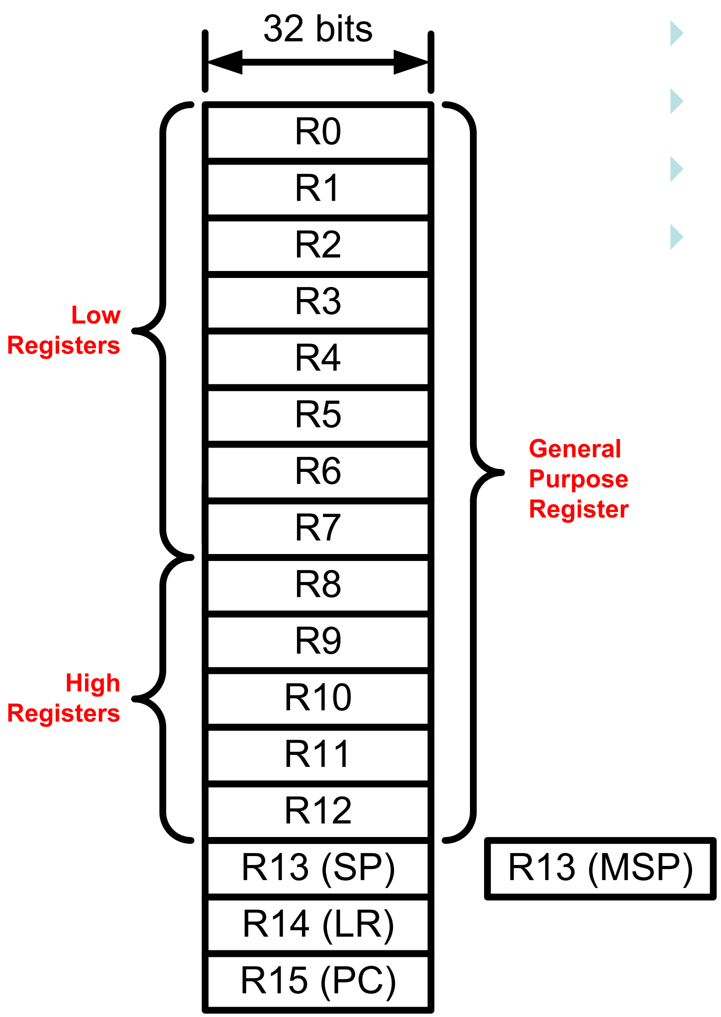
  ```
  <Operation>{<cond>}{S} Rd, Rn, Operand2
  ```

- Addition
  - ADD, ADC (add with carry)
- Subtraction
  - SUB, RSB (reverse subtract), SBC (subtract with carry)
- Multiplication
  - MUL (multiply), MLA (multiply-accumulate), MLS (multiply-subtract), SMULL (signed long multiply-accumulate), SMLAL (signed long multiply-accumulate), UMULL (unsigned long multiply-subtract), UMLAL (unsigned long multiply-subtract)
- Division
  - SDIV (signed), UDIV (unsigned)
- Logic
  - AND (bitwise and), ORR (bitwise or), EOR (bitwise exclusive or), ORN (bitwise or not), MVN (move not)

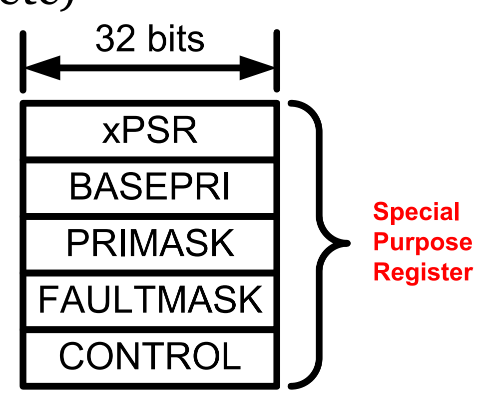
Overview: Arithmetic and Logic Instructions

- Bit set/clear
  - BFC (bit field clear), BFI (bit field insert), BIC (bit clear), CLZ (count leading zeroes)
- Bit/byte reordering
  - RBIT (reverse bit order in a word), REV (reverse byte order in a word), REV16 (reverse byte order in each half-word independently), REVSH (Reverse byte order in bottom half-word and sign extend.)
- Shift
  - LSL (logic shift left), LSR (logic shift right), ASR (arithmetic shift right), ROR (rotate right), RRX (rotate right with extend)
- Saturation
  - SSAT (signed), USAT (unsigned)
- Sign extension
  - SXTB (signed), SXTH, UXTB, UXTH
- Bit field extract
  - SBFX (signed), UBFX (unsigned)

Processor Registers



- Fastest way to read and write
- Registers are within the processor chip
- A register stores 32-bit value
- STM32L has
  - **R0-R12**: 13 general-purpose registers
  - **R13**: Stack pointer (Shadow of MSP or PSP)
  - **R14**: Link register (LR)
  - **R15**: Program counter (PC)
  - Special registers (xPSR, BASEPRI, PRIMASK, etc)



Example: Add

– Unified Assembler Language (UAL) Syntax

```
ADD r1, r2, r3     ; r1 = r2 + r3
ADD r1, r2, #4     ; r1 = r2 + 4
```

– Traditional Thumb Syntax

```
ADD r1, r3         ; r1 = r1 + r3
ADD r1, #15        ; r1 = r1 + 15
```

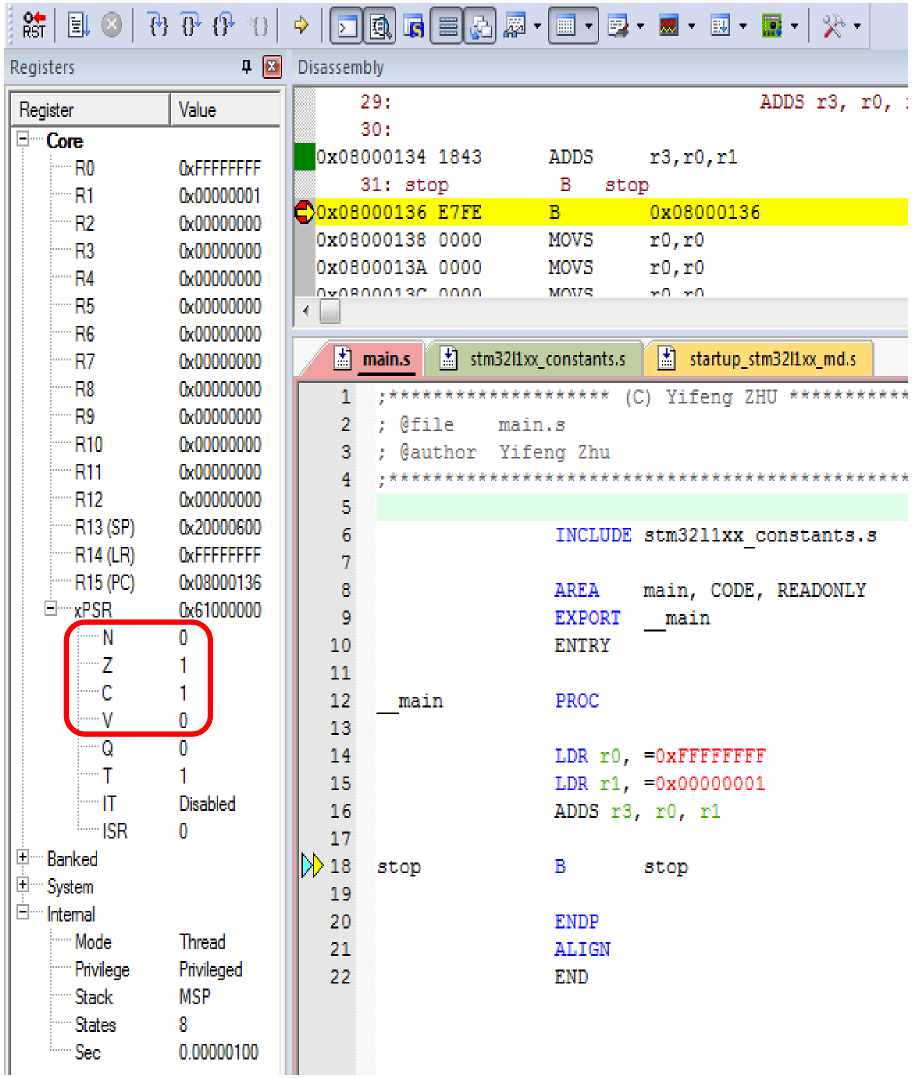
Commonly Used Arithmetic Operations

ADD {Rd,} Rn, Op2	Add. Rd ← Rn + Op2
ADC {Rd,} Rn, Op2	Add with carry. Rd ← Rn + Op2 + Carry
SUB {Rd,} Rn, Op2	Subtract. Rd ← Rn - Op2
SBC {Rd,} Rn, Op2	Subtract with carry. Rd ← Rn - Op2 - Carry
RSB {Rd,} Rn, Op2	Reverse subtract. Rd ← Op2 - Rn
MUL {Rd,} Rn, Rm	Multiply. Rd ← (Rn × Rm)[31:0]
MLA Rd, Rn, Rm, Ra	Multiply with accumulate. Rd ← (Ra + (Rn × Rm))[31:0]
MLS Rd, Rn, Rm, Ra	Multiply and subtract, Rd ← (Ra – (Rn × Rm))[31:0]
SDIV {Rd,} Rn, Rm	Signed divide. Rd ← Rn / Rm
UDIV {Rd,} Rn, Rm	Unsigned divide. Rd ← Rn / Rm
SSAT Rd, #n, Rm {,shift #s}	Signed saturate
USAT Rd, #n, Rm {,shift #s}	Unsigned saturate

Example:
S: Set Condition Flags

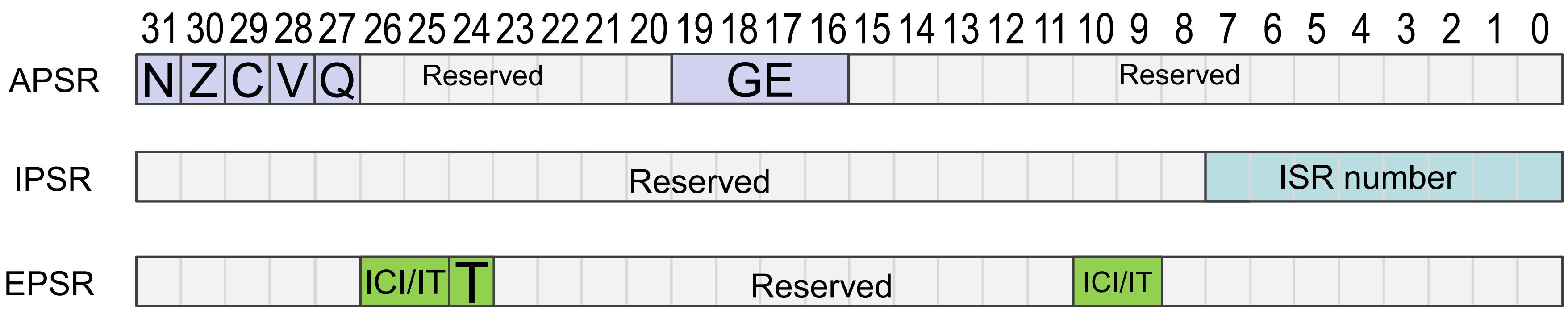
```
start
            LDR         r0, =0xFFFFFFFF
            LDR         r1, =0x00000001
            ADDS        r0, r0, r1
stop        B           stop
```

- For most instructions, we can add a suffix **S** to update the NZCV bits of the APSR register.
- In this example, the Z and C bits are set.



Program Status Register

- Application PSR (**APSR**), Interrupt PSR (**IPSR**), Execution PSR (**EPSR**)

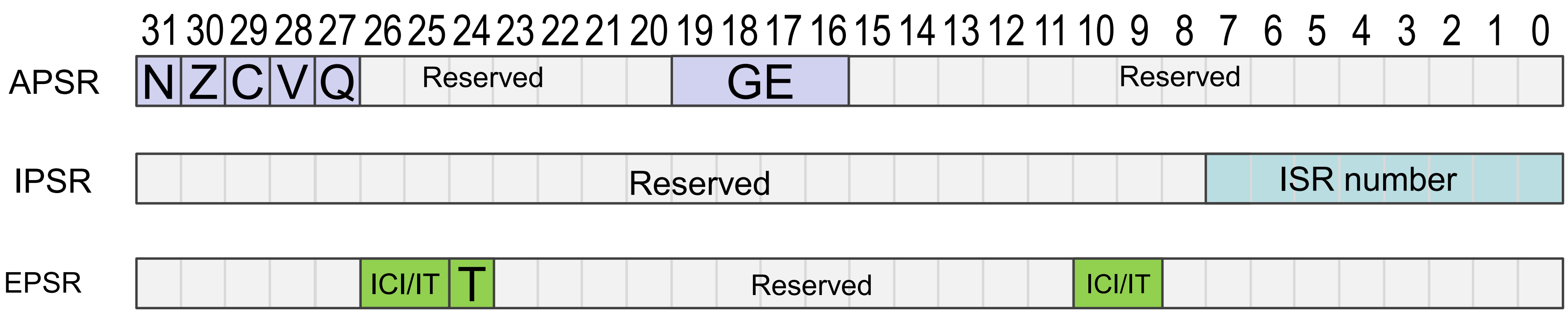


Note:

- GE flags are only available on Cortex-M4 and M7

Program Status Register

- Application PSR (**APSR**), Interrupt PSR (**IPSR**), Execution PSR (**EPSR**)



Combine them together into one register (**PSR**)

Note:

- GE flags are only available on Cortex-M4 and M7
- Use PSR in code

Example: 64-bit Addition

	Most-significant (Upper) 32 bits								Least-significant (Lower) 32 bits							
	0	0	0	0	0	0	0	2	F	F	F	F	F	F	F	F
+	0	0	0	0	0	0	0	4	0	0	0	0	0	0	0	1
	0	0	0	0	0	0	0	7 ←	0	0	0	0	0	0	0	0

Carry out

- A register can only store 32 bits
- A 64-bit integer needs two registers
- Split 64-bit addition into two 32-bit additions

Example: 64-bit Addition

```
start
  ; C = A + B
  ; Two 64-bit integers A (r1,r0) and B (r3, r2).
  ; Result C (r5, r4)
  ; A = 00000002FFFFFFFF
  ; B = 0000000400000001
  LDR  r0, =0xFFFFFFFF  ;  A's lower 32 bits
  LDR  r1, =0x00000002  ;  A's upper 32 bits
  LDR  r2, =0x00000001  ;  B's lower 32 bits
  LDR  r3, =0x00000004  ;  B's upper 32 bits

  ;  Add A and B using two instructions



stop  B  stop
```

Example: 64-bit Addition

```
start
  ; C = A + B
  ; Two 64-bit integers A (r1,r0) and B (r3, r2).
  ; Result C (r5, r4)
  ; A = 00000002FFFFFFFF
  ; B = 0000000400000001
  LDR  r0, =0xFFFFFFFF   ;  A's lower 32 bits
  LDR  r1, =0x00000002   ;  A's upper 32 bits
  LDR  r2, =0x00000001   ;  B's lower 32 bits
  LDR  r3, =0x00000004   ;  B's upper 32 bits

  ;  Add A and B
  ADDS r4, r2, r0   ; C[31..0] = A[31..0] +  B[31..0],  update Carry
  ADC  r5, r3 r1    ; C[64..32] = A[64..32] + B[64..32]  +  Carry

stop  B  stop
```

Example: Short Multiplication and Division

```
; MUL: Signed multiply
MUL  r6, r4, r2         ; r6 = LSB32( r4 × r2 )

; UMUL: Unsigned multiply
UMUL r6, r4, r2         ; r6 = LSB32( r4 × r2 )

; MLA: Multiply with accumulation
MLA  r6, r4, r1, r0     ; r6 = LSB32( r4 × r1 ) + r0

; MLS:  Multiply with subtract
MLS  r6, r4, r1, r0     ; r6 = LSB32( r4 × r1 ) - r0
```

Example: Long Multiplication

UMULL RdLo, RdHi, Rn, Rm	Unsigned long multiply. RdHi,RdLo ← unsigned(Rn × Rm)
SMULL RdLo, RdHi, Rn, Rm	Signed long multiply. RdHi,RdLo ← signed(Rn × Rm)
UMLAL RdLo, RdHi, Rn, Rm	Unsigned multiply with accumulate. RdHi,RdLo ← unsigned(RdHi,RdLo + Rn × Rm)
SMLAL RdLo, RdHi, Rn, Rm	Signed multiply with accumulate. RdHi,RdLo ← signed(RdHi,RdLo + Rn × Rm)

```
UMULL r3, r4, r0, r1   ; r4:r3 = r0 × r1, r4 = MSB bits, r3 = LSB bits
SMULL r3, r4, r0, r1   ; r4:r3 = r0 × r1
UMLAL r3, r4, r0, r1   ; r4:r3 = r4:r3 + r0 × r1
SMLAL r3, r4, r0, r1   ; r4:r3 = r4:r3 + r0 × r1
```

Bitwise Logic

AND {Rd,} Rn, Op2	Bitwise logic AND. Rd ← Rn & operand2
ORR {Rd,} Rn, Op2	Bitwise logic OR. Rd ← Rn | operand2
EOR {Rd,} Rn, Op2	Bitwise logic exclusive OR. Rd ← Rn ^ operand2
ORN {Rd,} Rn, Op2	Bitwise logic NOT OR. Rd ← Rn | (NOT operand2)
BIC {Rd,} Rn, Op2	Bit clear. Rd ← Rn & NOT operand2
BFC Rd, #lsb, #width	Bit field clear. Rd[(width+lsb–1):lsb] ← 0
BFI Rd, Rn, #lsb, #width	Bit field insert. Rd[(width+lsb–1):lsb] ← Rn[(width-1):0]
MVN Rd, Op2	Move NOT, logically negate all bits. Rd ← 0xFFFFFFFF EOR Op2

Example: AND r2, r0, r1

32 bits

r0	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
r1	1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 1
r2	1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

Bit-wise Logic AND

Example: ORR r2, r0, r1

32 bits

r0	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
r1	1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 1
r2	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Bit-wise Logic OR

Example: BIC r2, r0, r1

Bit Clear

r2 = r0 & NOT r1

Step 1:

r1	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1
NOT r1	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0

Step 2:

r0	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
NOT r1	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0
r2	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0

Example: BFC and BFI

- Bit Field Clear (BFC) and Bit Field Insert (BFI).
- Syntax
  - `BFC Rd, #lsb, #width`
  - `BFI Rd, Rn, #lsb, #width`

- Examples:

```
BFC R4, #8, #12
```

; Clear bit 8 to bit 19 (12 bits) of R4 to 0

```
BFI R9, R2, #8, #12
```

; Replace bit 8 to bit 19 (12 bits) of R9 with bit 0 to bit 11 from R2.

Example: SBFX and UBFX

- Signed and Unsigned Bit Field Extract.
- Copies adjacent bits from one register into the least significant bits of a second register, and sign extends or zero extends to 32 bits.
- Syntax
  - `SBFX Rd, RN, #lsb, #width`
  - `UBFX Rd, Rn, #lsb, #width`

- Examples:

```
SBFX R4, R5, #8, #12
```

; Copy bit 8 to bit 19 (12 bits) of R5 to bit 0 to bit 11 of R4 and sign extend it

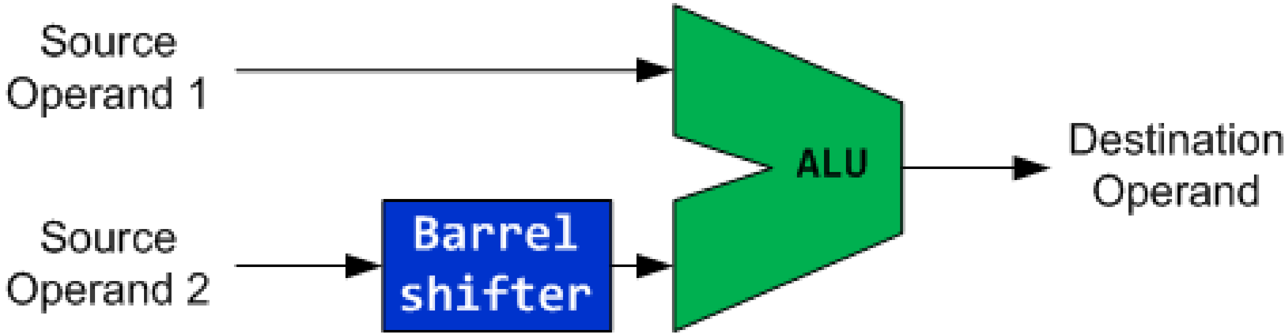
```
UBFX R9, R2, #8, #12
```

; Replace bit 8 to bit 19 (12 bits) of R2 to bit 0 to bit 11 from R9 and zero extend it.

Comparison Instructions

- Comparisons produce no results - just set condition codes.
- CMP like SUBS (Compare)
- CMN like ADDS (subtract of a negative number is the same as add) (Compare Negative)
- TST like ANDS (Test Bits)
- TEQ like EORS (eor of identical numbers gives result of zero) (Test Equivalence)

Barrel Shifter

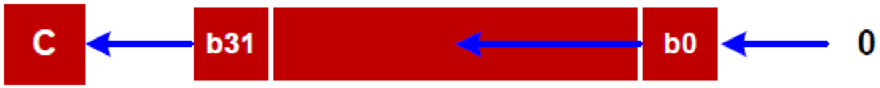


- The second operand of ALU has a special hardware called Barrel shifter
- Example:

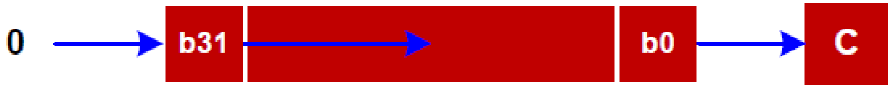
```
ADD r1, r0, r0, LSL #3  ; r1 = r0 + r0 << 3 = 9 × r0
```

The Barrel Shifter

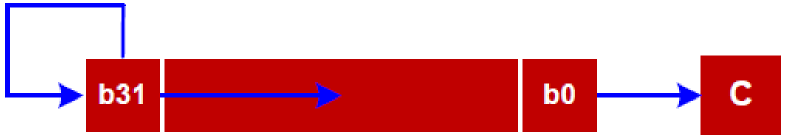
Logical Shift Left (**LSL**)



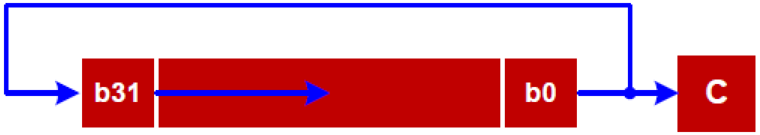
Logical Shift Right (**LSR**)



Arithmetic Shift Right (**ASR**)



Rotate Right (**ROR**)



Why is there rotate right but no rotate left?

Barrel Shifter

- Examples:
  - `ADD r1, r0, r0, LSL #3`

    `; r1 = r0 + r0 << 3 = r0 + 8 × r0`
  - `ADD r1, r0, r0, LSR #3`

    `; r1 = r0 + r0 >> 3 = r0 + r0/8 (unsigned)`
  - `ADD r1, r0, r0, ASR #3`

    `; r1 = r0 + r0 >> 3 = r0 + r0/8 (signed)`

- Use Barrel shifter to speed up the application

```
ADD r1, r0, r0, LSL #3  <=>  MOV r2, #9      ; r2 = 9
                             MUL r1, r0, r2 ; r1 = r0 * 9
```

Sign and Zero Extension

```
signed int_8  a = -1;   // a signed 8-bit integer,  a = 0xFF
signed int_16 b = -2;   // a signed 16-bit integer, b = 0xFFFE
signed int_32 c;        // a signed 32-bit integer

c = a;                  // sign extension required, c = 0xFFFFFFFF
c = b;                  // sign extension required, c = 0xFFFFFFFE
```

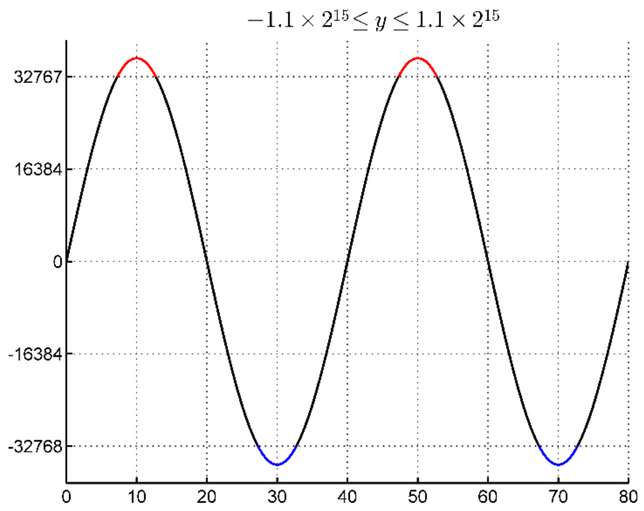
Sign and Zero Extension

SXTB {Rd,} Rm {,ROR #n}	Sign extend a byte. Rd[31:0] ← Sign Extend((Rm ROR (8 × n))[7:0])
SXTH {Rd,} Rm {,ROR #n}	Sign extend a half-word. Rd[31:0] ← Sign Extend((Rm ROR (8 × n))[15:0])
UXTB {Rd,} Rm {,ROR #n}	Zero extend a byte. Rd[31:0] ← Zero Extend((Rm ROR (8 × n))[7:0])
UXTH {Rd,} Rm {,ROR #n}	Zero extend a half-word. Rd[31:0] ← Zero Extend((Rm ROR (8 × n))[15:0])

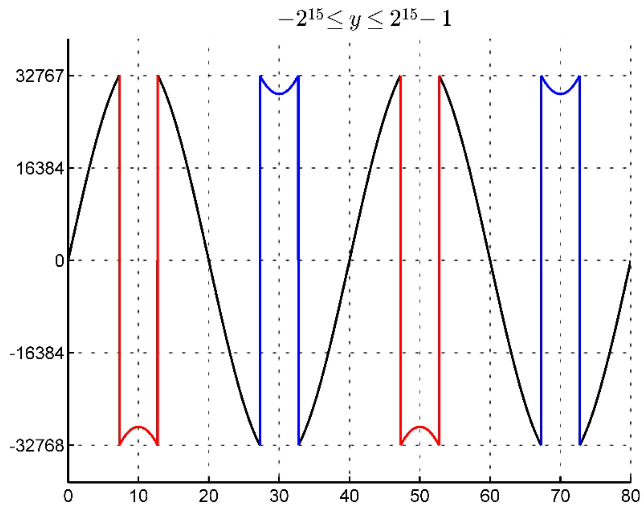
```
LDR R0, =0x55AA8765
SXTB R1, R0     ; R1 = 0x00000065
SXTH R1, R0     ; R1 = 0xFFFF8765
UXTB R1, R0     ; R1 = 0x00000065
UXTH R1, R0     ; R1 = 0x00008765
```

Example of Saturation

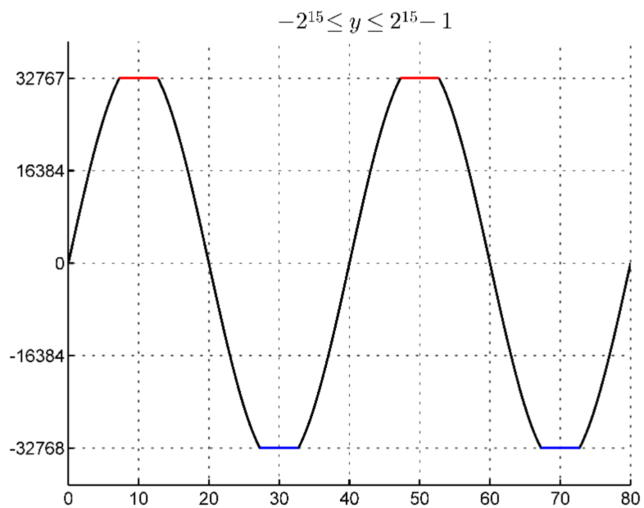
Assume data are limited to **16** bits



Without saturation



With saturation



Saturating Instruction: SSAT and USAT

- Syntax:
  - op{cond} Rd, #n, Rm{, shift}
- SSAT saturates a signed value to the signed range $-2^{n-1} \leq x \leq 2^{n-1} - 1$.

$$SAT(x) = \begin{cases} 2^{n-1} - 1 & if\ x > 2^{n-1} - 1 \\ -2^{n-1} & if\ x < 2^{n-1} \\ x & otherwise \end{cases}$$

- USAT saturates a signed value to the unsigned range $0 \leq x \leq 2^n - 1$.

$$USAT(x) = \begin{cases} 2^n - 1 & if\ x > 2^n - 1 \\ x & otherwise \end{cases}$$

- Examples:
  - `SSAT r2, #11, r1    ; output range:` $-2^{10} \leq r2 \leq 2^{10}$
  - `USAT r2, #11, r3    ; output range:` $0 \leq r2 \leq 2^{11}$

Reverse Order

RBIT Rd, Rn	Reverse bit order in a word. for (i = 0; i < 32; i++) Rd[i] ← RN[31– i]
REV Rd, Rn	Reverse byte order in a word. Rd[31:24] ← Rn[7:0], Rd[23:16] ← Rn[15:8], Rd[15:8] ← Rn[23:16], Rd[7:0] ← Rn[31:24]
REV16 Rd, Rn	Reverse byte order in each half-word. Rd[15:8] ← Rn[7:0], Rd[7:0] ← Rn[15:8], Rd[31:24] ← Rn[23:16], Rd[23:16] ← Rn[31:24]
REVSH Rd, Rn	Reverse byte order in bottom half-word and sign extend. Rd[15:8] ← Rn[7:0], Rd[7:0] ← Rn[15:8], Rd[31:16] ← Rn[7] & 0xFFFF

RBIT Rd, Rn

Rn	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Rd	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

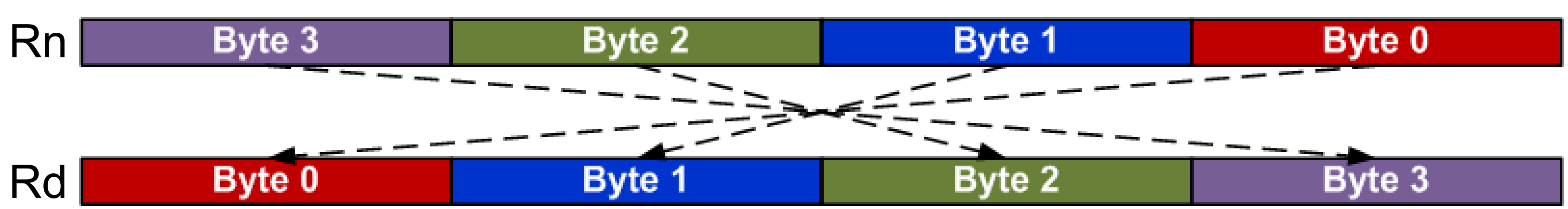
Example:

```
LDR  r0, =0x12345678  ; r0 = 0x12345678
RBIT r1, r0           ; Reverse bits, r1 = 0x1E6A2C48
```

Reverse Order

RBIT Rd, Rn	Reverse bit order in a word. for (i = 0; i < 32; i++) Rd[i] ← RN[31– i]
REV Rd, Rn	Reverse byte order in a word. Rd[31:24] ← Rn[7:0], Rd[23:16] ← Rn[15:8], Rd[15:8] ← Rn[23:16], Rd[7:0] ← Rn[31:24]
REV16 Rd, Rn	Reverse byte order in each half-word. Rd[15:8] ← Rn[7:0], Rd[7:0] ← Rn[15:8], Rd[31:24] ← Rn[23:16], Rd[23:16] ← Rn[31:24]
REVSH Rd, Rn	Reverse byte order in bottom half-word and sign extend. Rd[15:8] ← Rn[7:0], Rd[7:0] ← Rn[15:8], Rd[31:16] ← Rn[7] & 0xFFFF

REV Rd, Rn



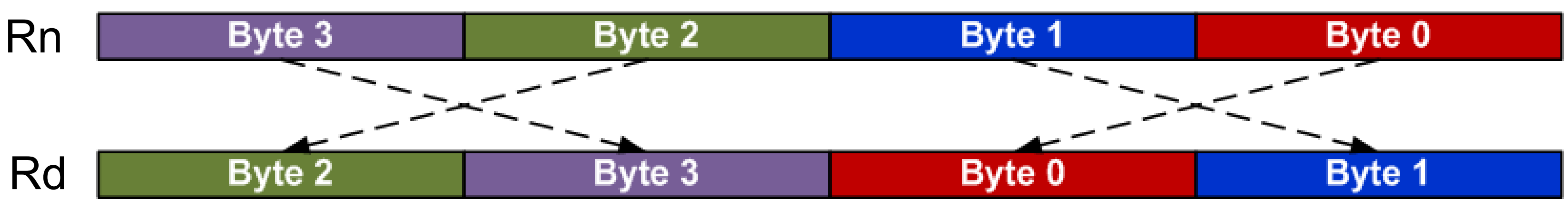
Example:

```
LDR R0, =0x12345678
REV R1, R0     ; R1 = 0x78563412
```

Reverse Order

RBIT Rd, Rn	Reverse bit order in a word. for (i = 0; i < 32; i++) Rd[i] ← RN[31– i]
REV Rd, Rn	Reverse byte order in a word. Rd[31:24] ← Rn[7:0], Rd[23:16] ← Rn[15:8], Rd[15:8] ← Rn[23:16], Rd[7:0] ← Rn[31:24]
REV16 Rd, Rn	Reverse byte order in each half-word. Rd[15:8] ← Rn[7:0], Rd[7:0] ← Rn[15:8], Rd[31:24] ← Rn[23:16], Rd[23:16] ← Rn[31:24]
REVSH Rd, Rn	Reverse byte order in bottom half-word and sign extend. Rd[15:8] ← Rn[7:0], Rd[7:0] ← Rn[15:8], Rd[31:16] ← Rn[7] & 0xFFFF

REV16 Rd, Rn



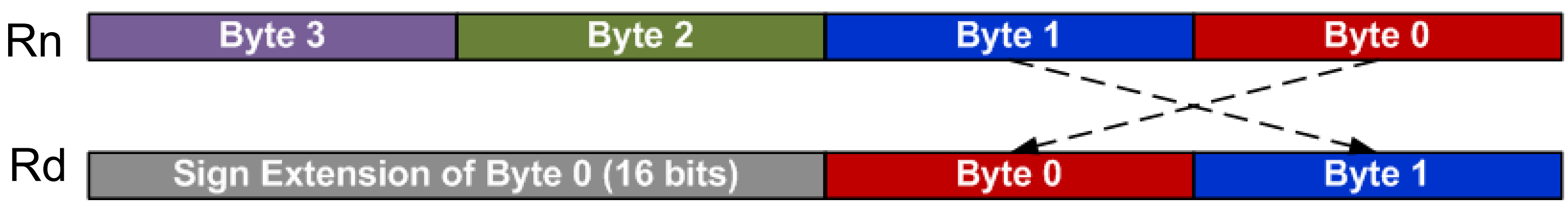
Example:

```
LDR R0, =0x12345678
REV16 R2, R0   ; R2 = 0x34127856
```

Reverse Order

RBIT Rd, Rn	Reverse bit order in a word. for (i = 0; i < 32; i++) Rd[i] ← RN[31– i]
REV Rd, Rn	Reverse byte order in a word. Rd[31:24] ← Rn[7:0], Rd[23:16] ← Rn[15:8], Rd[15:8] ← Rn[23:16], Rd[7:0] ← Rn[31:24]
REV16 Rd, Rn	Reverse byte order in each half-word. Rd[15:8] ← Rn[7:0], Rd[7:0] ← Rn[15:8], Rd[31:24] ← Rn[23:16], Rd[23:16] ← Rn[31:24]
REVSH Rd, Rn	Reverse byte order in bottom half-word and sign extend. Rd[15:8] ← Rn[7:0], Rd[7:0] ← Rn[15:8], Rd[31:16] ← Rn[7] & 0xFFFF

REVSH Rd, Rn



Example:

```
LDR R0, =0x33448899
REVSH R1, R0   ; R0 = 0xFFFF9988
```

Data Movement Between Registers

MOV Rd, oprand2	Rd ← operand2
MVN Rd, oprand2	Rd ← NOT operand2
MRS Rd, spec_reg	Move from **special** register to general register
MSR spec_reg, Rm	Move from **general** register to special register

```
MOV r4, r5              ; Copy r5 to r4
MVN r4, r5              ; r4 = bitwise logical NOT of r5
MOV r1, r2, LSL #3      ; r1 = r2 << 3
MOV r0, PC              ; Copy PC (r15) to r0
MOV r1, SP              ; Copy SP (r14) to r1
```